



---

# Islamic University of Technology

Department of Computer Science and Engineering (CSE)

B.Sc. in Software Engineering

**SWE 4802: Software Maintenance Lab**

Summer, 2024-2025

---

## Lab 4

Reverse Engineering

Namisa Najah Raisa

210042112

Program: B.Sc in SWE

Department of CSE

Date: July 16, 2026

# Contents

|          |                               |          |
|----------|-------------------------------|----------|
| <b>1</b> | <b>Reverse Engineering</b>    | <b>2</b> |
| 1.1      | IDA tool . . . . .            | 2        |
| 1.1.1    | Features of IDA Pro . . . . . | 3        |
| 1.2      | Doxygen . . . . .             | 8        |
| 1.2.1    | Reengineering . . . . .       | 9        |

# 1 Reverse Engineering

## 1.1 IDA tool

1. Install IDA tool
2. create a c test file
3. run *cl test.c* with the developer command prompt to create .exe and .obj files.
4. open IDA
5. open the *test.exe* file. This is the exe file IDA disassembled.

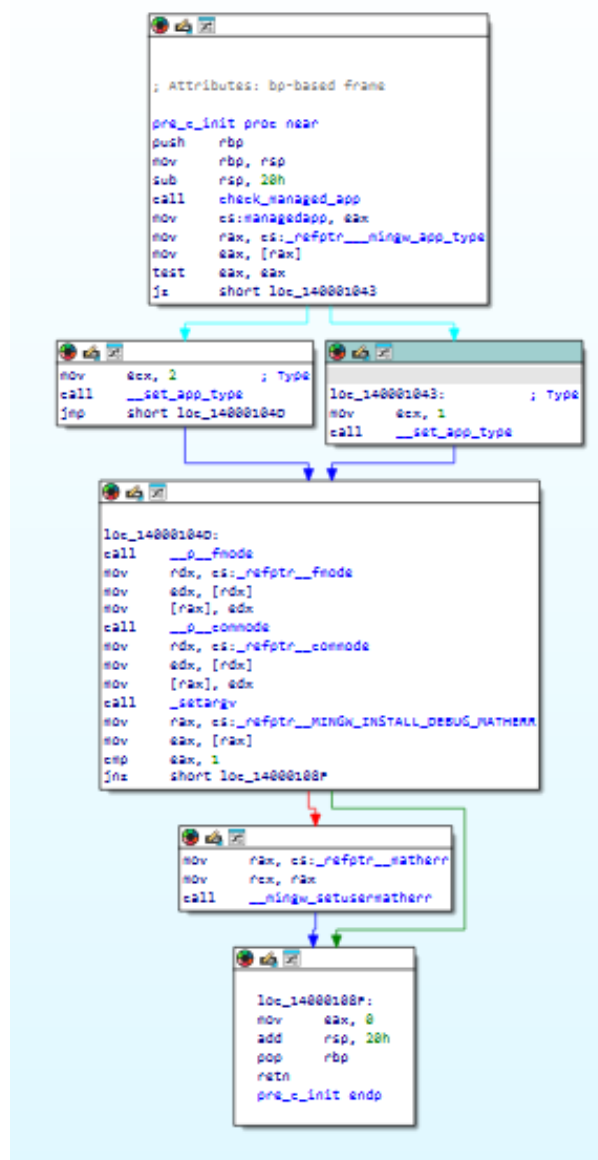


Figure 1: Disassembly of Add graph view

### 1.1.1 Features of IDA Pro

1. **Disassembly view:** Automatically converts machine code into human-readable assembly instructions.
  - (a) Open any executable in IDA; the main disassembly listing (IDA View-A) appears automatically after auto-analysis finishes.
2. **Graph view:** Displays a function's control flow as connected blocks instead of linear text, useful for visualizing branches and loops.

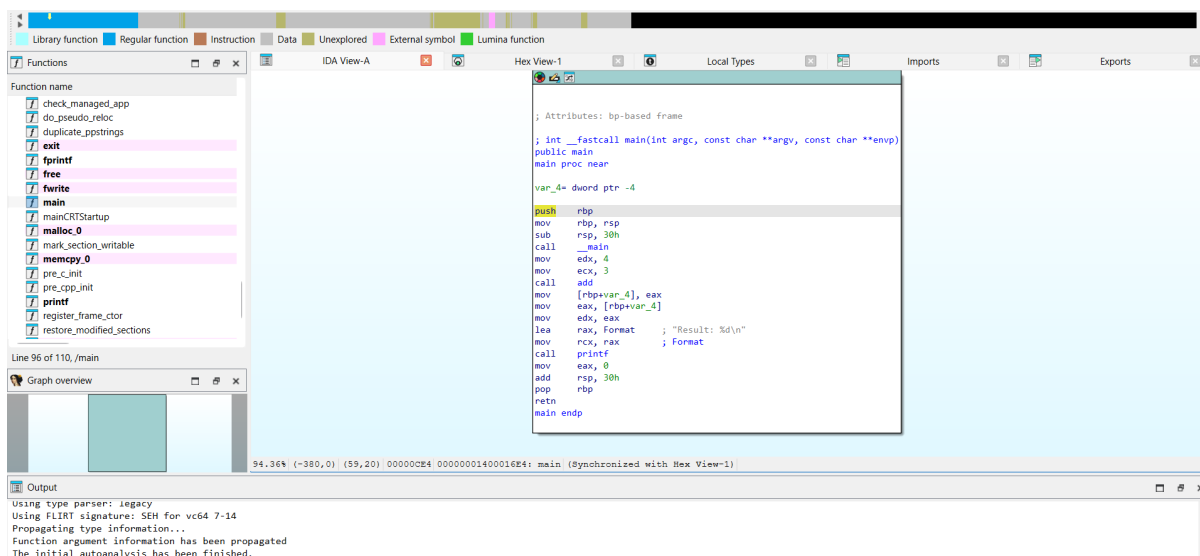


Figure 2: Disassembly of Main graph view

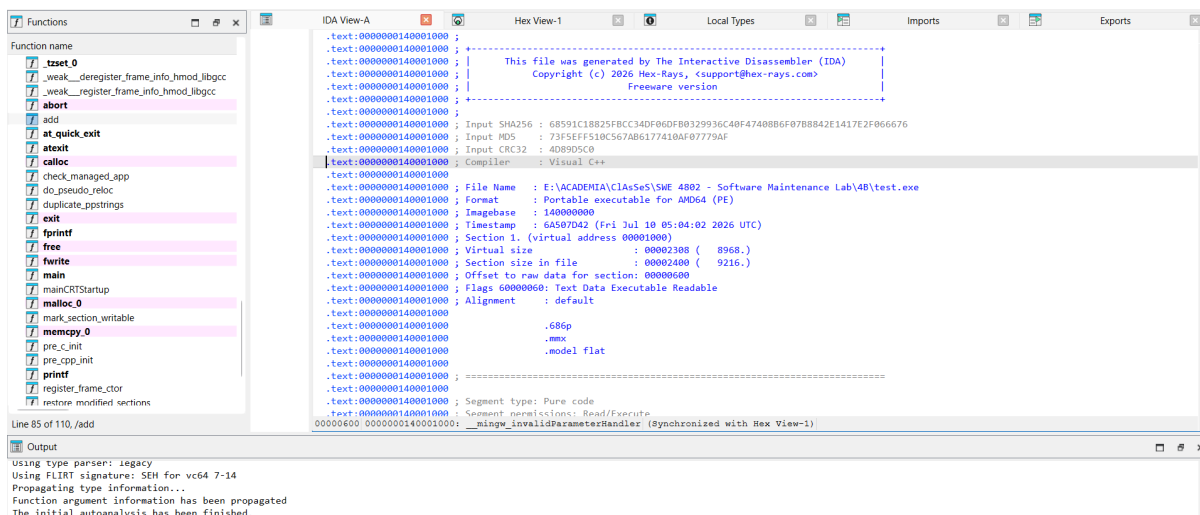


Figure 3: Disassembly of Main text view

- (a) Click into a function's disassembly (e.g. *main*).
  - (b) Press *Spacebar* to toggle between text view and graph view.
3. **Functions window:** Lists every function IDA identified in the binary, color-coded by type (library, regular, imported, etc.).

- (a) Located in the left panel by default.
- (b) Double-click any function name (e.g. *add*) to jump directly to its disassembly.

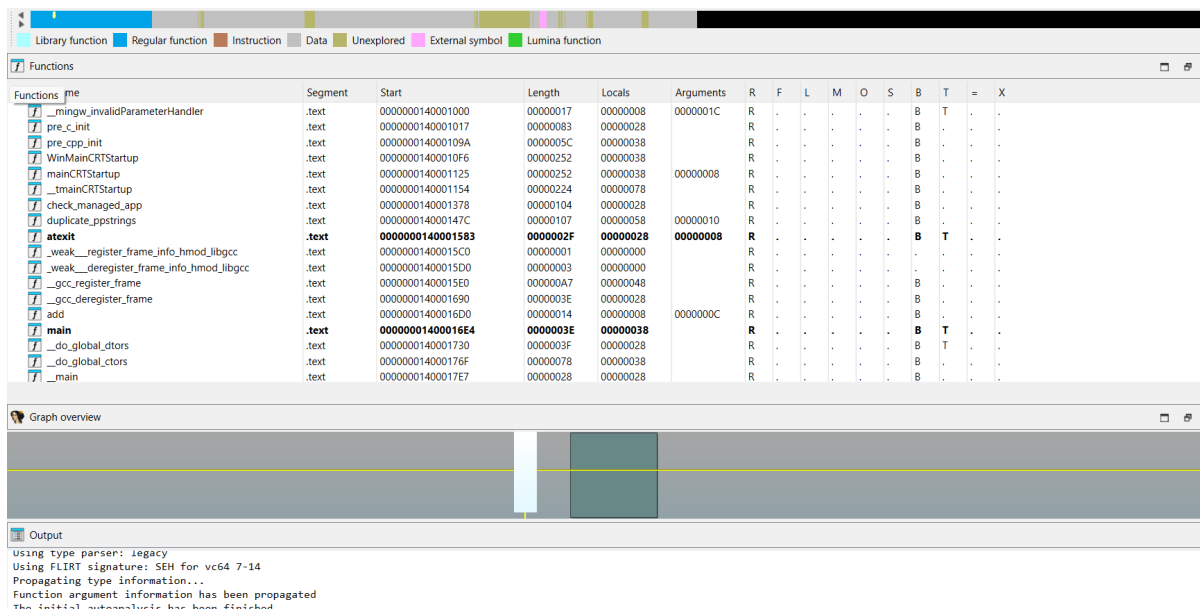


Figure 4: Functions window

4. **Cross-references (Xrefs):** Shows every location that calls, jumps to, or references a given function or variable.

- (a) Click on a function name (e.g. *add*).
- (b) Press *Ctrl+X*, or right-click → *Jump to xref to operand*.
- (c) IDA lists every call site - confirms, for example, that *add* is only called from *main*.

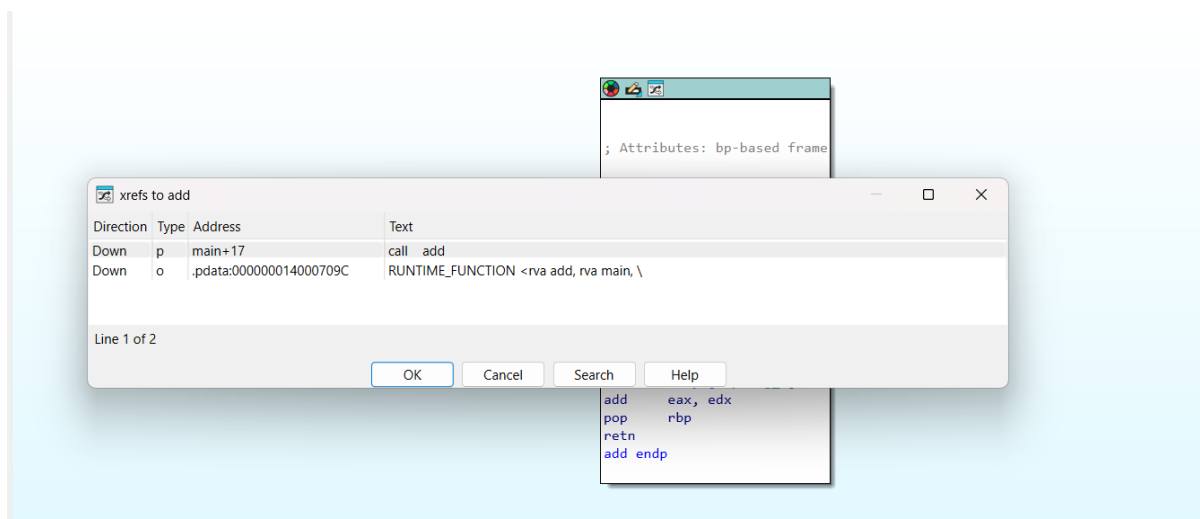


Figure 5: Xrefs

5. **Renaming:** Lets the analyst assign meaningful names to variables and functions for clarity.

- (a) Right-click a variable (e.g. `var_4`) → *Rename*.
- (b) Type a descriptive name (e.g. `result`) and confirm.

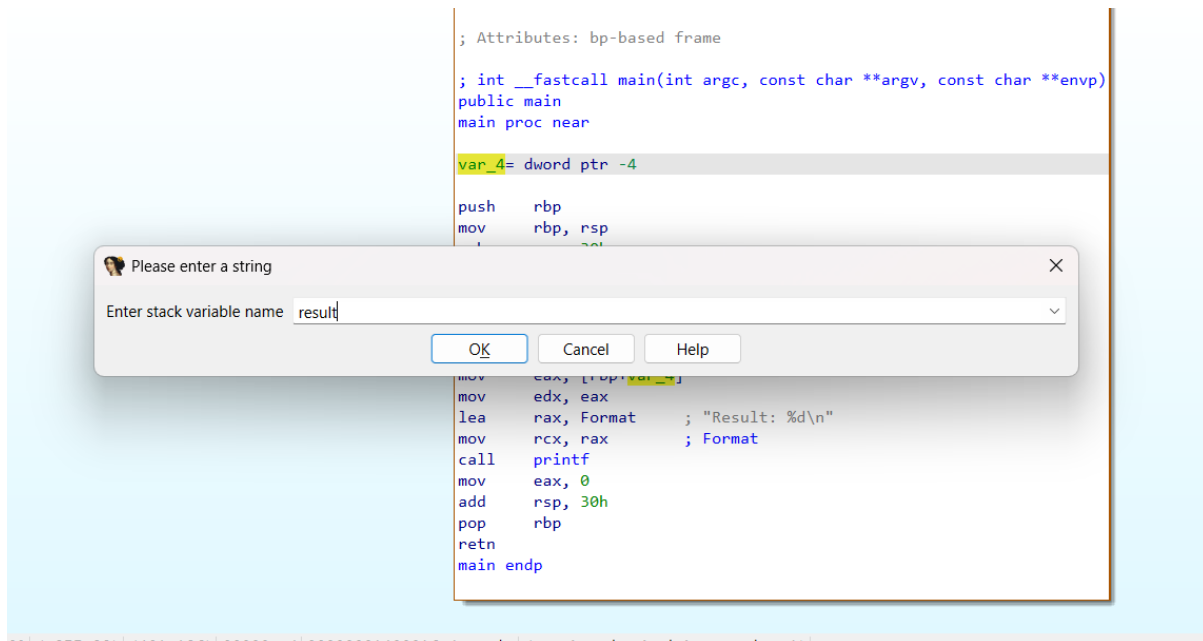
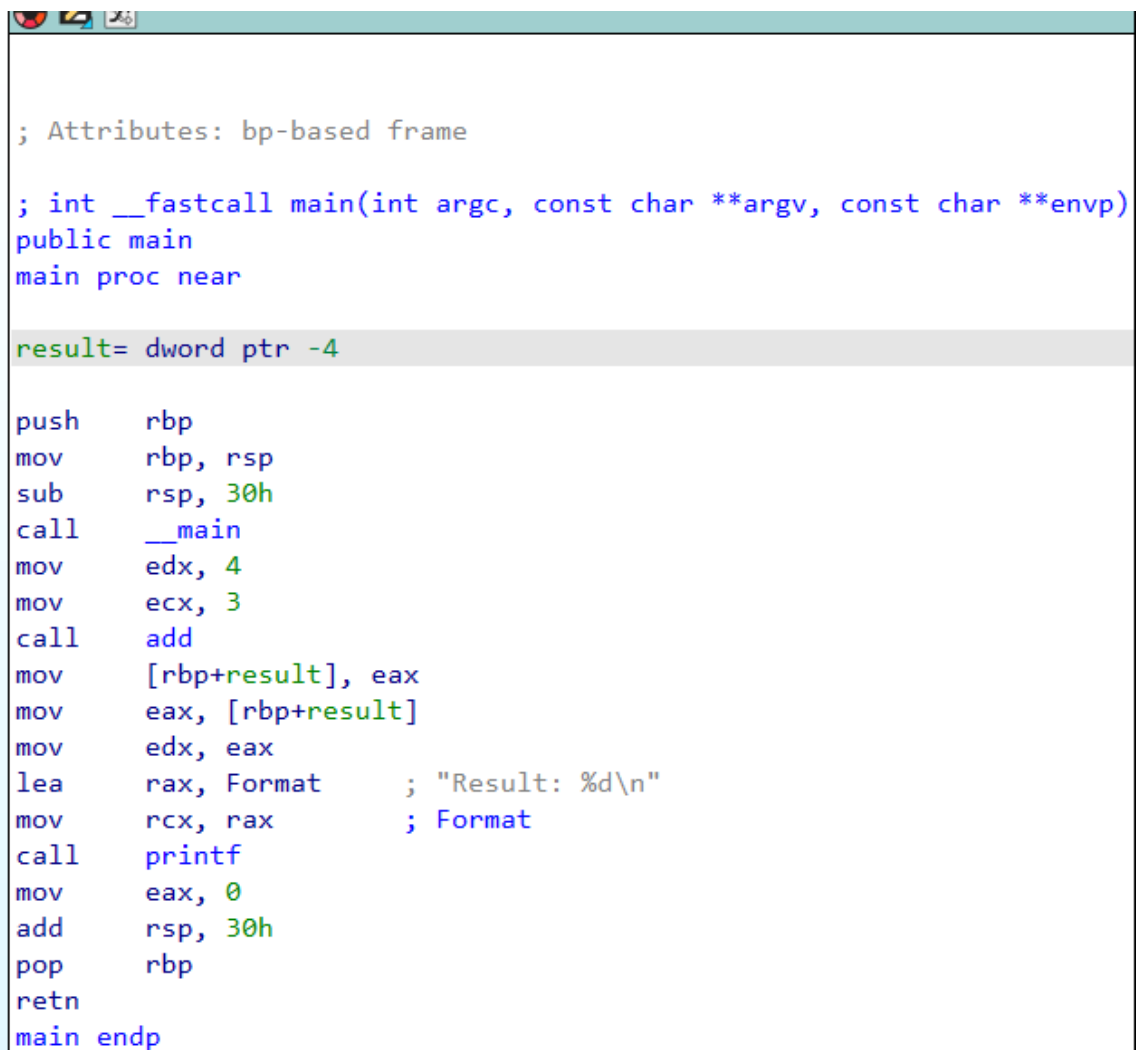


Figure 6: Renaming the `var_4` variable



```
; Attributes: bp-based frame

; int __fastcall main(int argc, const char **argv, const char **envp)
public main
main proc near

result= dword ptr -4

push    rbp
mov     rbp, rsp
sub     rsp, 30h
call    __main
mov     edx, 4
mov     ecx, 3
call    add
mov     [rbp+result], eax
mov     eax, [rbp+result]
mov     edx, eax
lea     rax, Format      ; "Result: %d\n"
mov     rcx, rax         ; Format
call    printf
mov     eax, 0
add     rsp, 30h
pop     rbp
retn
main endp
```

Figure 7: The var\_4 variable is now renamed to result

6. **Hex View:** Displays the raw bytes of the executable, synchronized with the disassembly cursor.
- (a) Open automatically alongside IDA View-A as *Hex View-1*.
  - (b) Clicking an instruction in the disassembly highlights its corresponding raw bytes.

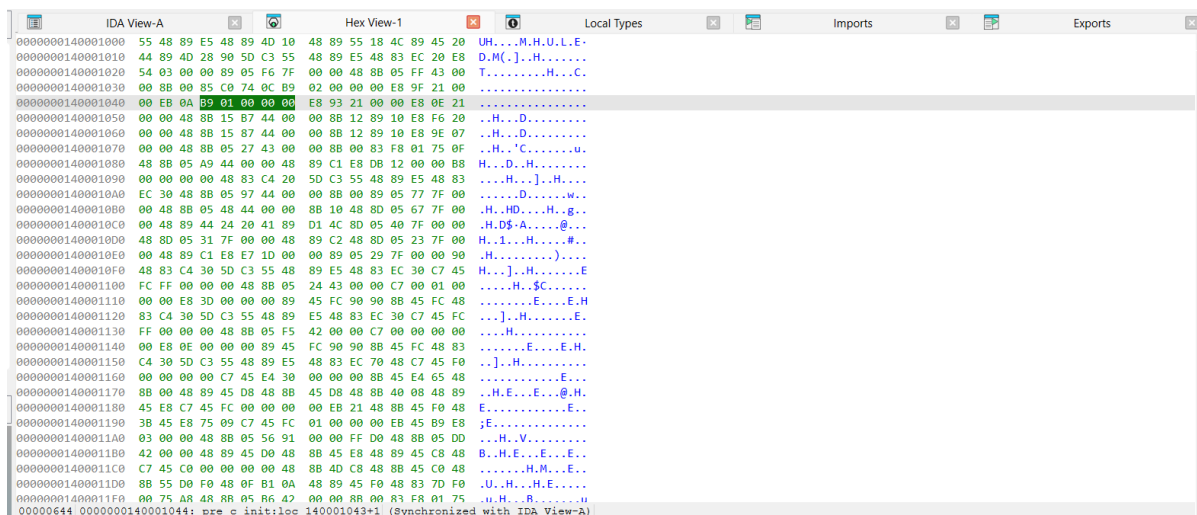


Figure 8: Hex view of the Add function

7. **Imports/Exports tabs:** Shows external library functions the binary calls (Imports) and functions the binary exposes (Exports).

(a) Click the *Imports* tab to see functions like *printf* resolved from the C runtime DLL.

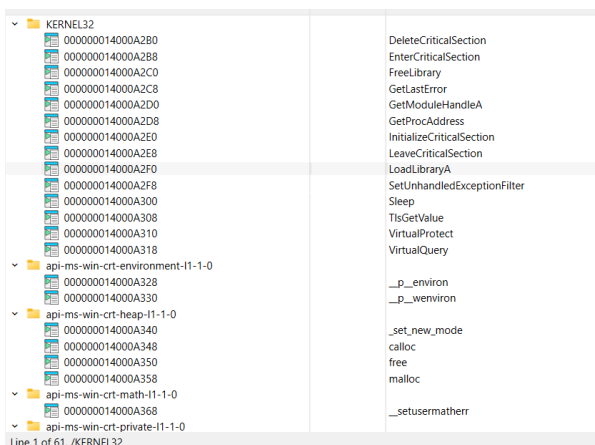


Figure 9: Imports

8. **Symbol stripping comparison:** Demonstrates the value of symbol information by analyzing a stripped binary.

(a) Compile a stripped version: `cl /Fe:test_stripped.exe test.c` then strip symbols, or compile with a flag that omits debug info.

(b) Open *test\_stripped.exe* in IDA.

(c) Observe that function names like *add* are replaced with generic labels such as *sub\_140001410*.

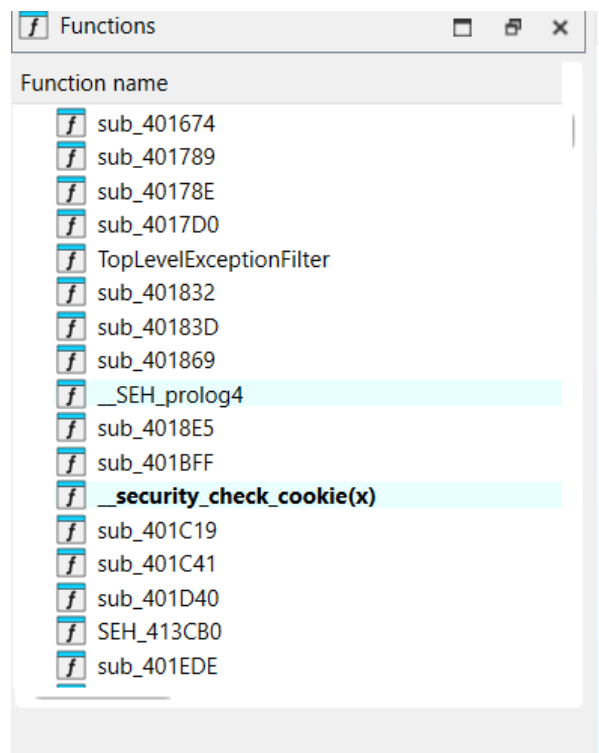


Figure 10: Stripped names

## 1.2 Doxygen

1. Install doxygen.
2. Install graphviz.
3. Java JDK 17+ needed.
4. Find a java codebase. I used a small library system.
5. Create a config file using *doxygen -g Doxyfile*
6. Edit the doxyfile.

```
1 PROJECT_NAME      = "Library Demo"
2 INPUT              = src
3 RECURSIVE          = YES
4 GENERATE_HTML      = YES
5 OPTIMIZE_OUTPUT_JAVA = YES
6 EXTRACT_ALL        = YES
7 HAVE_DOT           = YES
8 CALL_GRAPH         = YES
9 CALLER_GRAPH       = YES
10 CLASS_GRAPH        = YES
11 COLLABORATION_GRAPH = YES
```

Figure 11: Config file contents

7. Run *doxygen Doxyfile* to generate the documentation.
8. Open *html/index.html* in a browser which is the "recovered" documentation showing classes, methods, and relationships.



Figure 12: A fraction of the documentation and visualization with graphviz

### 1.2.1 Reengineering

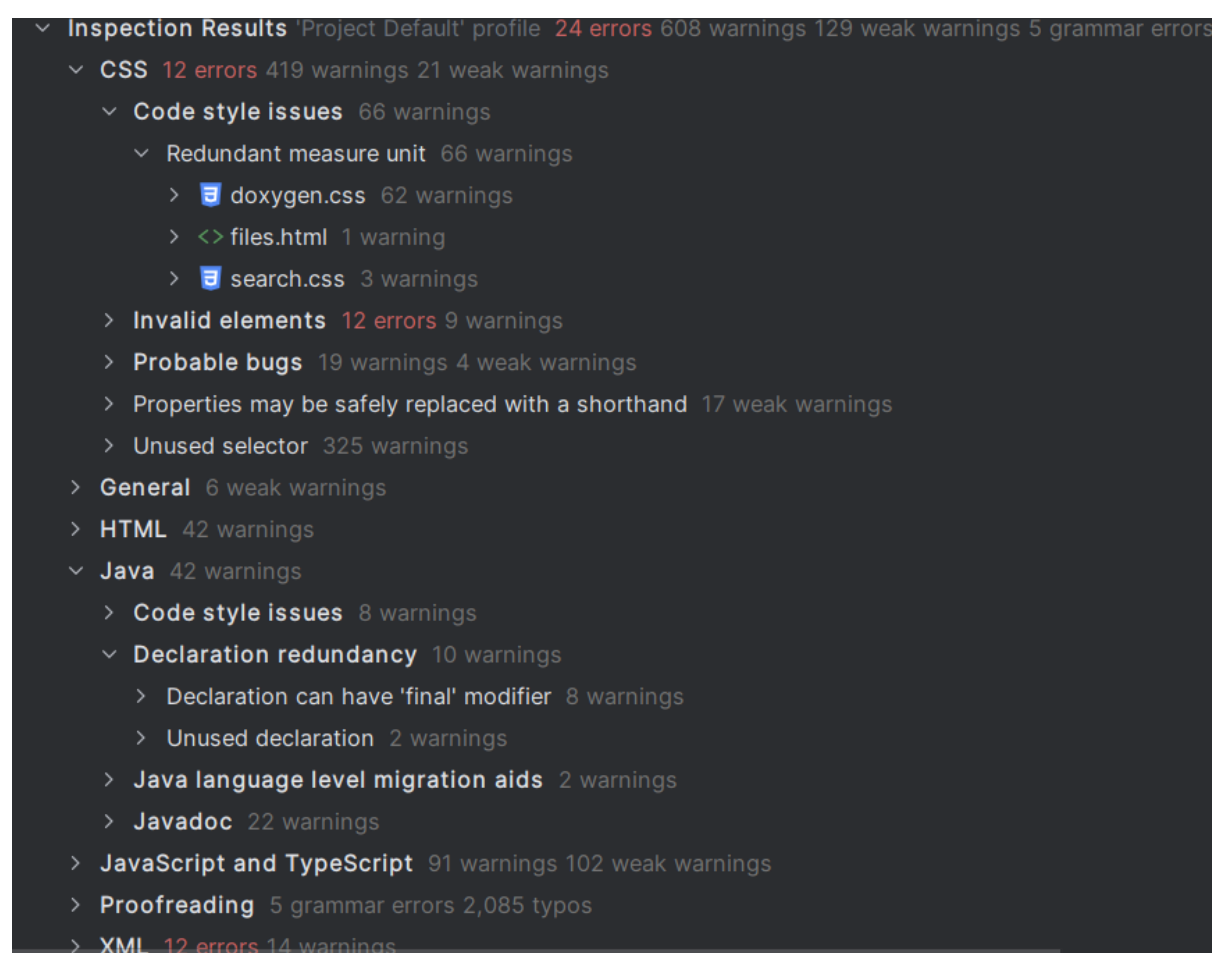


Figure 13: Java project analysis